SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – Case Study

Course Code:	DSA0305	**Course Title:**	Natural Language Processing
CO Assessed:	CO3 – Precision	**Assessment:**	Assessment Tool 2 – Case Study
Weightage:	40% of CIA		
CO5 Statement:	Apply N-gram models, smoothing techniques, part-of-speech tagging systems and to evaluate effective syntactic analysis. (BL-3)		
Student Name:	P.nagendra	**Reg. No.:**	192472399
Section / Batch:	CSE AI	**Date:**	08 / 08 / 2026

Code of Conduct

I __nagendra/192472399____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: ___Rohan V_____

Instructions

- Read all three case studies carefully before attempting the questions.
- Provide appropriate justifications and interpretations for every computed result.
- Use NLP concepts, algorithms, and terminology wherever applicable.
- Diagrams, flowcharts, tables, or mathematical formulations may be used to support your answers.
- Duration: 30 minutes.

Section / Topic	Focus	Case Study
N-Gram Language Models and Smoothing Techniques	Bigram and Trigram probability computation, Maximum Likelihood Estimation (MLE), Backoff models, Deleted Interpolation, Entropy calculation, uncertainty analysis, real-time next-word prediction	Case Study 1 – Smart Mobile Keyboard Prediction System
Part-of-Speech (POS) Tagging and Word Classes	Penn Treebank tagset, contextual ambiguity resolution, rule-based tagging, stochastic tagging (HMM), emission and transition probabilities, word class identification, chatbot language understanding	Case Study 2 – AI-Powered Customer Support Chatbot
Transformation-Based Tagging (Brill Tagger)	POS tag correction, transformation rules, contextual tagging, linguistic validation, tag sequence refinement, ambiguity handling	Case Study 3 – News Analytics and POS Tag Correction System
Corpus Statistics and Probabilistic Analysis	Word frequency counting, corpus-based probability estimation, entropy-based uncertainty measurement, tagging confidence evaluation	Case Study 3 – News Analytics and POS Tag Correction System

Integrated Syntactic Analysis and NLP Applications	Application of language models, POS tagging, probabilistic methods, corpus analysis, ambiguity resolution, and decision-making in real-world NLP systems	Case Studies 1, 2, and 3

CASE STUDY 1: Smart Mobile Keyboard Prediction System

A smartphone company is developing an AI-powered keyboard application that predicts the next word while users type emails, messages, and social media posts. The language model is trained on the corpus:

"Data science is powerful, data science drives innovation, data science is evolving."

The development team employs Bigram and Trigram Language Models, Backoff Models, Deleted Interpolation, and Entropy Analysis to improve prediction accuracy and handle unseen word sequences.

The following statistics are obtained from the corpus:

- C(data) = 3
- C(data science) = 3
- C(science is) = 2
- C(science drives) = 1
- C(science drives innovation) = 1

The interpolation weights are:

- λ_1 = 0.5 (Trigram)
- λ_2 = 0.3 (Bigram)
- λ_3 = 0.2 (Unigram)

The prediction probabilities after the word "science" are:

- P(is) = 0.66
- P(drives) = 0.33

Questions

1. Compute the Maximum Likelihood Estimate (MLE) of the Bigram Probability P(science | data). Interpret how this probability influences next-word prediction in a mobile keyboard application.
2. A user types the unseen sequence "data science improves". Apply the Backoff Model and estimate the probability using lower-order n-grams. Explain why backoff is essential for real-time predictive text systems.
3. Using Deleted Interpolation, calculate the probability of the sequence "data science is". Discuss how interpolation balances reliability and coverage in sparse datasets.
4. Calculate the Entropy of the distribution P(is)=0.66 and P(drives)=0.33. Interpret the result and evaluate how entropy helps measure uncertainty and prediction confidence in intelligent keyboard systems.

CASE STUDY 2: AI-Powered Customer Support Chatbot

A multinational airline deploys a customer support chatbot that processes user messages and automatically identifies the grammatical role of words before generating responses.

Consider the following user inputs:

1. "Book a flight ticket now."
2. "This book is interesting."

The chatbot uses the Penn Treebank POS Tagset and a Hidden Markov Model (HMM) for probabilistic tagging.

Given:

- P(book | VB) = 0.6
- P(book | NN) = 0.4
- P(Start → VB) = 0.5
- P(Start → NN) = 0.5

The system must determine whether the word "book" functions as a Verb (VB) or Noun (NN) depending on context.

Questions

1. Assign appropriate Penn Treebank POS tags to every word in both sentences. Justify the tag assigned to the word "book" using contextual clues.
2. Using the HMM probabilities, compute the likelihood of "book" being tagged as a Verb (VB) in the sentence "Book a flight ticket now." Explain why the probabilistic model favors this tag.
3. Compare Rule-Based Tagging and Stochastic (HMM) Tagging in resolving lexical ambiguity. Recommend which approach is more suitable for large-scale chatbot deployment and justify your answer.
4. Evaluate the role of standardized POS tagsets and word classes in improving intent detection, response generation, and overall chatbot performance.

CASE STUDY 3: News Analytics and POS Tag Correction System

A digital news analytics company processes millions of news articles daily. The platform uses a Transformation-Based Tagger (Brill Tagger) to improve tagging accuracy after an initial POS tagging phase.

The sentence processed by the system is:

"Economic growth increases employment."

Initial POS Tags:

- economic/JJ
- growth/NN
- increases/NNS
- employment/NN

A learned transformation rule states:

Change NNS to VBZ if the preceding word is tagged as NN.

The system also performs corpus-based frequency analysis and uncertainty evaluation.

Questions

1. Apply the transformation rule and update the POS tag sequence. Explain why the correction is linguistically appropriate.
2. Analyze the initial tagging errors and justify the corrected tag assignments using grammatical structure and sentence semantics.
3. Assuming the corpus contains the words:
    - economic (120 occurrences)
    - growth (450 occurrences)
    - increases (210 occurrences)
    - employment (380 occurrences)

    Compute the word frequency distribution and explain how frequency information supports probabilistic POS tagging.

4. Evaluate how Transformation-Based Tagging improves tagging accuracy compared with the initial tagging stage. Discuss how entropy can be used to measure uncertainty in tag assignments before and after rule application.

ANSWERS

CASE STUDY 1: Smart Mobile Keyboard Prediction System

Corpus: "Data science is powerful, data science drives innovation, data science is evolving." (12 word tokens, 7 distinct word types).

Given counts: C(data) = 3, C(data science) = 3, C(science is) = 2, C(science drives) = 1, C(science drives innovation) = 1. Interpolation weights: λ_1 = 0.5 (trigram), λ_2 = 0.3 (bigram), λ_3 = 0.2 (unigram).

Q1. MLE of the Bigram Probability P(science | data)

```
P(science | data) = C(data science) / C(data)
                  = 3 / 3
                  = 1.0
```

Interpretation: every time the word "data" appears in the training corpus, it is immediately followed by "science" so the maximum likelihood estimate is a full 1.0. For the keyboard, this means the model has complete confidence that "science" is the correct next-word suggestion whenever the user types "data", and it should surface that suggestion at the top of the prediction bar with no competing alternative. In practice a probability of exactly 1.0 from a tiny corpus is also a sign of data sparsity with more training data the estimate would likely settle to something below 1.0 once other continuations of "data" are observed.

Q2. Backoff Model for the unseen sequence "data science improves"

Backoff logic (Katz-style): use the highest-order n-gram that has been observed; if its count is zero, fall back to the next lower order, scaling by a discount/back-off weight α so the probabilities still sum to 1.

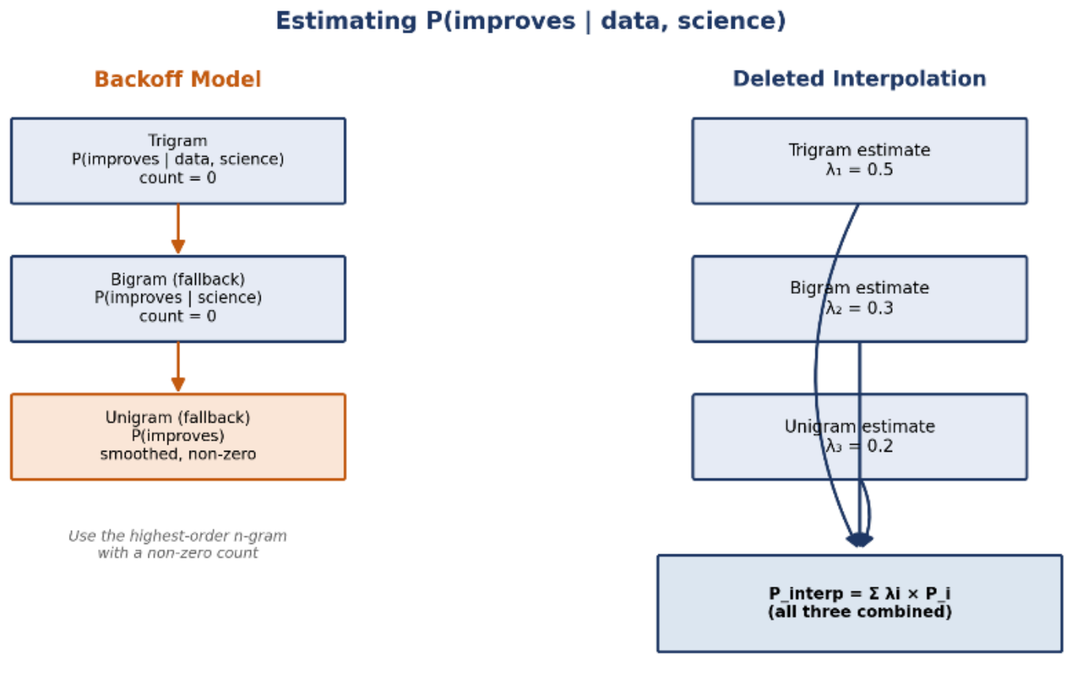
```
Step 1 (Trigram): P(improves | data, science)
      C(data science improves) = 0  ->  back off

Step 2 (Bigram):  P(improves | science)
      C(science improves) = 0  ->  back off further

Step 3 (Unigram): P(improves)
      C(improves) = 0 in this corpus
      -> assign a small smoothed probability instead of 0
        (e.g. via Laplace/Add-one or Katz discounting),
        roughly on the order of 1 / (N + V)
```

Since "improves" never occurs anywhere in this small training corpus, the raw counts at every level are zero. A pure backoff model would technically return 0 in that case, which is why real systems combine backoff with a discounting/smoothing method (Katz backoff, Kneser-Ney, etc.) that reserves a small amount of probability mass for unseen words rather than ruling them out completely.

Backoff is essential for real-time predictive text because users constantly type combinations that never appeared in the training data new topics, names, slang, or simply sequences too rare to have been observed. Without backoff, the model would assign zero probability to any unseen sequence and effectively refuse to make a prediction. By falling back to lower-order n-grams (and ultimately to smoothed unigram estimates), the keyboard can always



offer a reasonable, non-zero suggestion instead of failing silently.

Figure 1 — Backoff vs. Deleted Interpolation, side by side

Q3. Deleted Interpolation Probability of "data science is"

Deleted interpolation blends the trigram, bigram, and unigram MLE estimates using the given weights:

$$P_{interp}(is \mid data, science) = \lambda 1 \cdot P(is \mid data, science) + \lambda 2 \cdot P(is \mid science) + \lambda 3 \cdot P(is)$$

Component MLE estimates from the corpus:

$$\begin{aligned} P(is \mid data, science) &= C(data\ science\ is) / C(data\ science) = 2/3 = 0.667 \\ P(is \mid science) &= C(science\ is) / C(science) = 2/3 = 0.667 \\ P(is) &= C(is) / N = 2/12 = 0.167 \end{aligned}$$

$$\begin{aligned} P_{interp}(is \mid data, science) &= (0.5 \times 0.667) + (0.3 \times 0.667) + (0.2 \times 0.167) \\ &= 0.333 + 0.200 + 0.033 \\ &= 0.567 \end{aligned}$$

Chaining this with P(data) and P(science | data) gives the overall probability of the 3-word sequence:

$$\begin{aligned} P(data\ science\ is) &= P(data) \times P(science \mid data) \times P_{interp}(is \mid data, science) \\ &= 0.25 \times 1.0 \times 0.567 \\ &= 0.142 \end{aligned}$$

Interpolation balances reliability and coverage by never relying on a single order of n-gram: the trigram estimate is the most specific and informative when data is available, but it is also the most prone to sparsity (as seen with "data science improves" above, where the trigram count was 0). By blending it with the more frequently-observed bigram and unigram estimates, deleted interpolation keeps the specificity of higher-order context when the data supports it, while still producing sensible, non-zero probabilities in sparse or borderline cases which is exactly the situation a small, real-world training corpus is in most of the time.

Q4. Entropy of P(is) = 0.66, P(drives) = 0.33

$$\begin{aligned} H &= -\Sigma\, p(x) \cdot \log_2 p(x) \\ &= -[0.66 \cdot \log_2(0.66) + 0.33 \cdot \log_2(0.33)] \\ &= -[0.66 \times (-0.5995) + 0.33 \times (-1.5995)] \\ &= 0.3956 + 0.5278 \\ &= 0.923\ \text{bits} \end{aligned}$$

The maximum possible entropy for a two-outcome distribution is 1 bit, reached when both outcomes are equally likely (0.5/0.5). Here $H \approx 0.923$ bits is close to that maximum, which means the two candidate words "is" and "drives" are fairly close in probability the model is not strongly confident in a single next-word choice. For an intelligent keyboard, high entropy like this signals that the system should present both suggestions to the user rather than auto-committing to one, whereas a low-entropy distribution (one candidate close to 1.0) would justify aggressive auto-completion. Entropy therefore acts as a direct, quantitative measure of the keyboard's prediction confidence.

CASE STUDY 2: AI-Powered Customer Support Chatbot

Sentences: (1) "Book a flight ticket now." (2) "This book is interesting."

Given: P(book|VB) = 0.6, P(book|NN) = 0.4, P(Start→VB) = 0.5, P(Start→NN) = 0.5.

Q1. Penn Treebank POS Tags

Sentence	Tagged Sequence
Book a flight ticket now.	Book/VB a/DT flight/NN ticket/NN now/RB ./.
This book is interesting.	This/DT book/NN is/VBZ interesting/JJ ./.

Justification for "book": in sentence 1, "book" is the very first word of an imperative sentence and is immediately followed by a determiner + noun phrase ("a flight ticket") that acts as its object the classic structure of a command, so it is tagged VB (base-form verb). In sentence 2, "book" is preceded by the determiner "This" and followed by the verb "is", placing it in subject position of the sentence, so it is tagged NN (common noun). The surrounding context not the word form itself, since "book" is identical in both cases is what resolves the ambiguity.

Q2. HMM Likelihood of "book" as VB in "Book a flight ticket now."

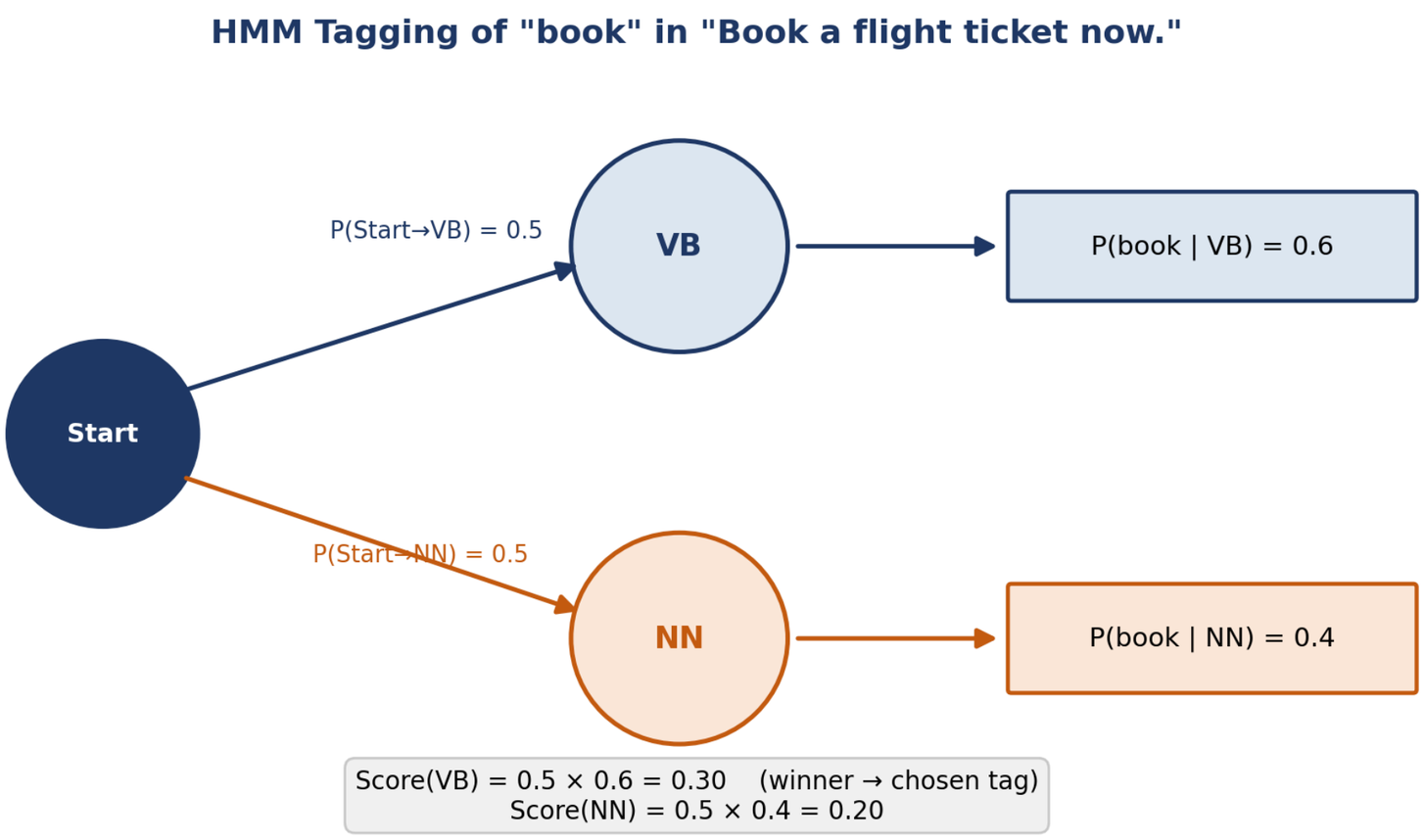
Since "book" is the first word of the sentence, its tag likelihood combines the initial (Start) transition probability with the emission probability of the word given that tag:

$$\text{Score}(VB) = P(\text{Start} \to VB) \times P(\text{book}|VB) = 0.5 \times 0.6 = 0.30$$
$$\text{Score}(NN) = P(\text{Start} \to NN) \times P(\text{book}|NN) = 0.5 \times 0.4 = 0.20$$

$$\text{Posterior: } P(VB|\text{book}, \text{Start}) = 0.30 / (0.30 + 0.20) = 0.6 \quad (60\%)$$
$$P(NN|\text{book}, \text{Start}) = 0.20 / (0.30 + 0.20) = 0.4 \quad (40\%)$$

The HMM assigns VB a joint score of 0.30 against NN's 0.20, so under the argmax (Viterbi) decision rule the tagger selects VB a 60% vs 40% posterior split. This happens because the emission probability P(book|VB) = 0.6 is higher than P(book|NN) = 0.4, and the two tags share the same starting transition probability, so the emission probability alone tips the decision toward the verb reading, which also happens to match the imperative structure of the sentence.

Figure 2 — HMM state diagram for tagging "book"



Q3. Rule-Based Tagging vs Stochastic (HMM) Tagging

Aspect	Rule-Based Tagging	Stochastic (HMM) Tagging

Basis	Hand-crafted linguistic rules and context patterns	Emission/transition probabilities learned from tagged corpora
Ambiguity handling	Resolves only cases the rule-writer anticipated	Resolves probabilistically, generalizes to unseen contexts
Maintenance	Requires manual rule updates as language evolves	Retrains automatically from new labeled data
Scalability	Becomes unwieldy as rule sets grow	Scales well to large vocabularies and domains
Transparency	Highly interpretable, easy to audit	Less transparent; decisions are statistical

For large-scale chatbot deployment where the system must handle millions of varied, evolving user messages across many topics stochastic (HMM) tagging is the more suitable approach, because it generalizes from data rather than requiring engineers to anticipate every ambiguous pattern by hand, and it scales far better as vocabulary and conversation styles grow. In practice, many production systems still layer a small set of rule-based corrections (similar to the Brill tagger in Case Study 3) on top of a statistical tagger to catch systematic errors combining the strengths of both.

Q4. Role of Standardized POS Tagsets and Word Classes

- Consistent tagging (e.g. Penn Treebank) gives every downstream module intent detection, entity extraction, response generation a common, unambiguous representation of sentence structure to work from.
- Correct word-class identification is often the deciding signal for intent detection: tagging "book" as VB versus NN is effectively the difference between a booking request and a reference to an object.
- Standardized tags support reliable slot filling (e.g. treating "flight ticket" as the object noun phrase of a booking intent), which feeds directly into structured backend queries.
- Knowing syntactic roles helps the response generator produce grammatically correct, natural replies rather than templated or awkward phrasing.
- Since customer-support language is full of homographs and polysemous words ("book", "charge", "return", "change"), reliable word-class tagging is critical to avoiding misclassification and downstream errors across multi-turn conversations.

CASE STUDY 3: News Analytics and POS Tag Correction System

Sentence: "Economic growth increases employment."

Initial tags: economic/JJ, growth/NN, increases/NNS, employment/NN.

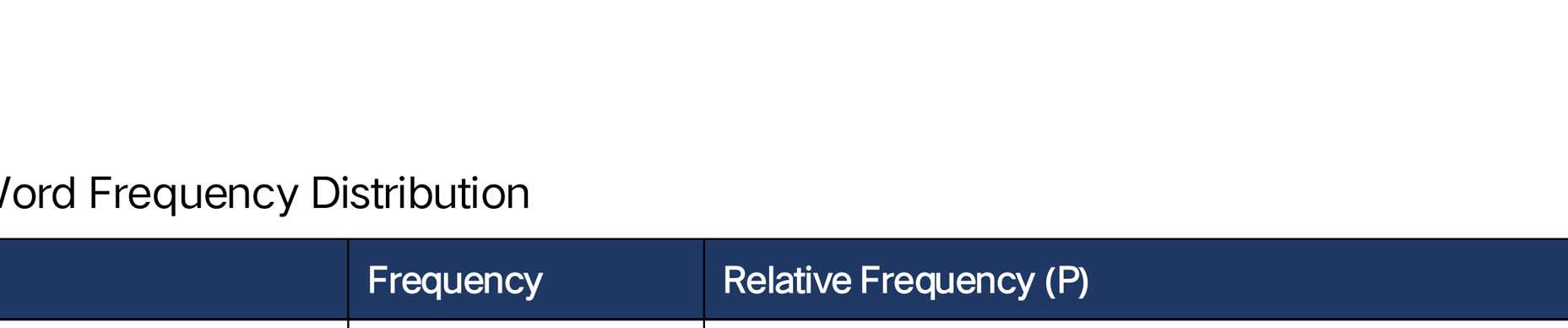
Learned transformation rule: change NNS to VBZ if the preceding word is tagged NN.

Q1. Applying the Transformation Rule

```
economic/JJ  growth/NN  increases/NNS  employment/NN     (initial)
                  |
   "growth" is tagged NN and immediately precedes "increases/NNS"
                  |
                  v
economic/JJ  growth/NN  increases/VBZ  employment/NN     (corrected)
```

The correction is linguistically appropriate because "increases" is functioning as the finite main verb of the sentence, agreeing in number with the singular subject "growth" (third-person singular present tense, hence VBZ rather than the base form VB). The initial tagger appears to have used the surface "-s" ending as a heuristic for pluralization and mistakenly tagged it as a plural noun (NNS), missing that "-s" also marks third-person singular verb agreement in English precisely the ambiguity this transformation rule is designed to catch.

Figure 3 — Transformation-based tagging pipeline



Q2. Analysis of the Initial Tagging Errors

The initial tagger relied on shallow, form-based cues (the "-s" suffix) rather than sentence-level structure, which is a common source of error for words that are ambiguous between plural nouns and third-person singular verbs (compare "increases", "changes", "reports" all can be either). Looking at the full sentence structure "Economic growth" (NP, subject) + "increases" + "employment" (NP, object) the sentence follows a canonical Subject – Verb – Object pattern. If "increases" were genuinely a plural noun, the sentence would have no finite verb at all and would be ungrammatical ("Economic growth increases employment" would just be two noun phrases with nothing connecting them). Semantically, too, the sentence describes growth causing a rise in employment an action relation that only a verb can express. Both the grammatical structure and the meaning of the sentence confirm that VBZ is the correct tag.

Q3. Word Frequency Distribution

Word	Frequency	Relative Frequency (P)
economic	120	120 / 1160 ≈ 0.103 (10.3%)
growth	450	450 / 1160 ≈ 0.388 (38.8%)
increases	210	210 / 1160 ≈ 0.181 (18.1%)
employment	380	380 / 1160 ≈ 0.328 (32.8%)
Total	1160	1.000 (100%)

These relative frequencies are exactly the kind of maximum-likelihood estimates that feed into probabilistic POS taggers: emission probabilities P(word|tag) and transition probabilities P(tag|previous tag) in an HMM are computed from counts just like these, drawn from a large annotated corpus. Words with high, stable frequency (like "growth" here) give the tagger more reliable statistical grounding, while rarer or more ambiguous words contribute weaker, noisier estimates which is exactly why supplementary transformation rules (as in Q1) are useful for catching the errors that pure frequency-based tagging misses.

Q4. Transformation-Based Tagging vs Initial Tagging, and the Role of Entropy

The initial tagging stage assigns tags independently based on surface form or simple statistical likelihood, which is fast but blind to certain contextual dependencies as shown by the NNS/VBZ confusion above. Transformation-Based Tagging (the Brill tagger) improves on this by learning an ordered sequence of contextual correction rules from a training corpus, applying each rule wherever its triggering context is met, and iteratively refining the tag sequence. This lets it fix systematic, context-dependent errors (like tagging a verb as a plural noun after a specific tag pattern) without needing to retrain a full probabilistic model.

Entropy is a useful way to quantify this improvement. Before the transformation rule is applied, if the tagger's evidence for "increases" is split between NNS and VBZ (for example, close to a 50/50 confidence split based on surface form alone), the entropy of that tag distribution is high near the 1-bit maximum for a two-way choice, exactly as computed for the keyboard case in Case Study 1. After the transformation rule deterministically reassigns the tag to VBZ based on strong contextual evidence (the preceding NN), the distribution collapses to essentially one certain outcome, and entropy drops close to 0 bits. This drop in entropy is a direct, measurable sign that Transformation-Based Tagging increases the model's confidence and, in turn, its accuracy.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly

Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Q. No. / Criteria Level	Understanding of Case	Application of Concepts	Analysis	Solution Design	Clarity	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____